C Programming

INDEX Previous Next

FUNCTIONS
A function in C can perform a particular task, and supports the concept of modular programming design techniques.

We have already been exposed to functions. The main body of a C program, identified by the keyword *main*, and enclosed by the left and right braces is a function. It is called by the operating system when the program is loaded, and when terminated, returns to the operating system.

Functions have a basic structure. Their format is

```
return_data_type  function_name  ( arguments, arguments )
data_type_declarations_of_arguments;
{
        function_body
}
```

It is worth noting that a return_data_type is assumed to be type *int* unless otherwise specified, thus the programs we have seen so far imply that *main()* returns an integer to the operating system.

ANSI C varies slightly in the way that functions are declared. Its format is

```
return_data_type function_name (data_type variable_name, data_type variab
{
        function_body
}
```

This permits type checking by utilizing function prototypes to inform the compiler of the type and number of parameters a function accepts. When calling a function, this information is used to perform type and parameter checking.

ANSI C also requires that the return_data_type for a function which does not return data must be type *void*. The default return_data_type is assumed to be integer unless otherwise specified, but must match that which the function declaration specifies.

A simple function is,

```
void print_message( void )
{
        printf("This is a module called print_message.\n");
}
```

Note the function name is *print_message*. No arguments are accepted by the function, this is indicated by the keyword *void* in the accepted parameter section of the function declaration. The return_data_type is void, thus data is not returned by t